

PyQCU CUDA-C++ Clover MultiGrid 系列开发报告

(git tag dev73 → dev73_5):

同步开销根因、Schur 一致粗空间与性能结果

ZhangXin

IMP, CAS

2026-08-03

摘要

本报告对象为 git tag dev73* 共六个版本标签(dev73,dev73_1,dev73_2,dev73_3,dev73_4,dev73_5), 系统总结 CUDA-C++ Clover MultiGrid 求解器 (applyCloverMultigridQcu) 从“正确但无性能优势”到**最高 $2.43\times$ 加速**的完整演进。报告基于 logs/ 下的全部开发/调试/优化/结果报告与 examples/qcu/ 下的全部测试/基准/剖析脚本, 给出**计算结果分析** (gauge 的 $SU(3)$ 性质、解误差、null_vecs 正确性、迭代次数) 与**计算性能分析** (耗时、加速比、热点分解、规模效应、参数扫描), 并以 **dev73_5** 为范例对各版本做**对照讨论**, 讲明从 dev73 到 dev73_5 的每一处改动、其原由与最终效果。

演进主线: dev73 (首个完整实现, 正确性 6.23×10^{-7} , 加速比仅 $0.87\text{--}1.28\times$) → dev73_2 (修 10 个根因 bug, level-0 改全算子, 正确但更慢 $0.15\text{--}0.41\times$) → dev73_3 (粗空间改基于 Schur 算子 S 自身零模、粗算子 $A_c = P^\top SP$ 物化为 33 张量 stencil, 主求解迭代 $\sim 90 \rightarrow 63\text{--}68$ 、减少约 30%, 但受 RTX 4060 内核启动延迟限制) → dev73_4 (定位 WSL2 同步开销 $\sim 177 \mu\text{s}$ /次为根因, 5 项同步极简优化, V100 上默认格子 $\{8, 16, 16, 16\}$ 2 层中位数加速比 $1.26\times$ 、 $8 \times 8 \times 8 \times 16$ 达 $2.07\times$) → dev73_5 (建立干净测量协议并做精度/格子/参数扩展扫描, $8 \times 8 \times 8 \times 16$ 复现 $2.43\times$ (c64) / $1.97\times$ (c128), 同时揭示加速比随格子规模增大而**下降且非单调**的规模效应)。全系列解与参考 BiStabCG 一致 ($vs_ref \sim 3 \times 10^{-7}$ c64 / 4×10^{-8} c128)。

目录

1 引言与范围	2
1.1 对象	2
1.2 数据来源	3
1.3 硬件环境	3
2 时间线与各 tag 改动概述	4
3 方法与符号	4
4 计算结果分析	5
4.1 Gauge 的 $SU(3)$ 性质	5

4.2	解误差与全残差	5
4.3	null_vecs 正确性	6
4.4	迭代次数演进	7
5	计算性能分析	8
5.1	各版本性能矩阵	8
5.2	性能演进	8
5.3	热点分解 (PROF_SECTIONS)	9
5.4	规模效应	9
5.5	精度扫描	10
5.6	参数扫描 (默认格子 c64 2L)	10
6	对照讨论 (以 dev73_5 为范例)	11
6.1	完整性能总表 (dev73_5)	11
6.2	各版本对照表	12
6.3	收敛历史对照	13
6.4	加速比与耗时对照 (dev73_5)	16
7	从 dev73 到 dev73_5: 改动、原由与最终效果	16
7.1	正确性主线	17
7.2	性能主线	17
7.3	工程主线	18
7.4	净变更统计	18
8	结论	18
A	图表清单 (所有数据图表)	19
B	数据文件清单	21
C	脚本文件清单 (examples/qcu)	21

1 引言与范围

1.1 对象

本报告的分析对象是 `git tag dev73*` 全部六个版本标签:

tag	提交	日期	一句话主题
dev73	bd81d31	2026-07-28	首个完整 C++ CUDA Clover MG (953 行)
dev73_1	cfbd753	2026-08-01	全库 CLAUDE.md 文档体系
dev73_2	d296612	2026-08-02	MG 系统调试 (10 个根因 bug) + 全算子 level-0
dev73_3	a1b9a95	2026-08-02	Schur 一致粗空间 + 33 张量粗算子
dev73_4	a724357	2026-08-02	同步开销根因 + 5 项同步极简优化
dev73_5	1955795	2026-08-03	干净测量协议 + 精度/格子/参数扩展扫描

表 1: dev73* 系列版本标签一览

1.2 数据来源

- **日志与报告** (logs/): dev73.md/.tex/.pdf、mg-debug-optimize-report-2026-08-01.md、multigrid-v2-report-2026-08-01.md、multigrid-v3-final-report-2026-08-01.md、multigrid-fix-schur-multigrid-report-2026-08-02.tex/.pdf、mg-v4-report-2026-08-02.tex/.pdf、mg-v4-fix-report-2026-08-02.tex/.pdf/.md 及其表格 (dev73_5_tbl_*.tex) 与图表 (dev73_5_*.png);
- **原始收敛/计时日志**: clover_multigrid.log、clover_multigrid_test.log、mg_bench_out.txt、mg_iter_sweep_out.txt、mg_param_sweep_out.txt、schur_mg_test_stdout.txt、pyref_*.log 等;
- **结构化结果**: multigrid_report.json、mg_dev_results.json、schur_mg_results.json、bench_report.json;
- **脚本** (examples/qcu/): conftest.clover.multigrid.py、bench_cpp_mg.py、mg_dev_*.py、v3_*.py、quick_test.py、verify_coarse.py、diag_coarse.py、mg_v4_*.py、mg_dev73_5_*.py (约 60 个脚本)。

1.3 硬件环境

系列开发期间硬件发生一次切换，是解释性能跳变的关键背景：

- **dev73 – dev73_3**: NVIDIA GeForce RTX 4060 Laptop (SM 8.9), WSL2。dev73_3 明确测得求解期间 GPU 稳定运行在 **210 MHz 空闲频率、利用率 2–5%**，即**内核启动延迟主导** (约 $107\ \mu\text{s}$ /次启动);
- **dev73_4 – dev73_5**: NVIDIA Tesla V100-SXM2-32GB (SM 7.0), WSL2。同步开销根因 (约 $177\ \mu\text{s}$ /次) 与性能结果均在此硬件测得。

2 时间线与各 tag 改动概述

tag	净变更 (vs 前一 tag)	主要内容
dev73	—	首个完整 C++ CUDA Clover MG (<code>lattice_clover_multigrid.h</code> 953 行, 5 流并行、 <code>device_vals</code> 标量管理、 <code>mpi_real_type<T></code> 精度模板) ; 三轮全库审查 135 项发现、33 个 bug 修复; 12 项性能优化; CLQCD 参考文档 dev71
dev73_1	43 文件 +1247	全库 CLAUDE.md 文档体系 (Python 包 / cpp 后端 / examples / docs/logs/refer), 清理旧 CMakeLists 符号链接, 无功能代码改动
dev73_2	45 文件 +5491/~451	MG 系统调试: 修 10 个根因 bug (prolong 转置索引、checkerboard 奇偶转换、粗修正缓冲被 clobber、指针按 float 而非复元素推进、全 site 网格尺寸、粗层 DOF、segfault 等); level-0 重写为 全算子 ; 粗层 sitting 用单位算子 + 受保护修正; 新增约 18 个调试/基准脚本与 4 份报告
dev73_3	40 文件 +8917/~367	Schur 一致粗空间 : 粗空间改基于 S 自身零模, $A_c = P^\top S P$ 物化为 33 张量 stencil (on-site+8 最近邻 +24 对角); 粗层性能 (cublasDot→单块 dot、单流轻量粗迭代、融合单 kernel 粗解); 修 V-cycle/数值 bug 与 2 个 Python 层 bug; 新增 <code>conftest.schur.multigrid.py</code> 与约 20 个脚本
dev73_4	18 文件 +1430/~35	定位 WSL2 同步开销根因 (~177 μ s/次); 5 项同步极简优化; 修 4 个 v4 bug; <code>null_vecs</code> 缓存与正确性验证; 参数扫描与最终基准
dev73_5	24 文件 +2012	结果扩展: 新增测量脚本族 <code>mg_dev73_5_*</code> , 干净测量协议 (独立进程 +ref/MG 交叉计时, min of 5 对), 精度/格子/参数扫描, 收敛/ 热点/加速比/时间/扫描图表与 LaTeX 表格

表 2: dev73 系列各 tag 净变更与主要内容

3 方法与符号

求解对象: 奇偶预条件 (Schur 补) Clover 系统

$$S x_o = b_o, \quad S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}, \quad (1)$$

参考求解器 = `applyCloverBistabCgQcu` (奇偶预条件 BiStabCG, `VERBOSE=0`) ; `MultiGrid` = `applyCloverMultigridQcu` (Schur 一致、多层粗层、33 张量粗算子、`null_vecs` 缓存)。

固定变量 (dev73_4/dev73_5): `mass=0.05`、`atol=10-6`、`gauge_seed=42`、 $\kappa = 1/(2 \cdot 0.05 + 8) \approx 0.1235$ 、`sigma=0.1`、`null_vecs` 迭代 `nv_iters=2`、`E=48`; 默认格子 {8,16,16,16} (各向均不小于)。

误差定义: $vs_{ref} = \|x_{mg} - x_{ref}\| / \|x_{ref}\|$ (MG 解与参考解相对差); $mg_{res} = \|b - D x_{mg}\| / \|b\|$ (MG 全残差)。

测量协议 (dev73_5, 干净测量): 每个配置在**独立进程**中完成; 参考 BiStabCG 与 MG **交叉计时** (ref, mg, ref, mg, ...), 各取 5 对的**最小值与中位数**, 最小化 GPU 时钟漂移影响。

热点: C++ 段计时 PROF_SECTIONS (fine_iter/vcycle/coarse_solve/coarse_dslash)。

4 计算结果分析

本节按 dev73 系列的**计算结果**(数值正确性)做系统分析。所有版本最终都收敛到与参考 BiStabCG 相同的解。

4.1 Gauge 的 SU(3) 性质

C++ Gauss 生成的 gauge 场(seed=42)经 `pyqcu.lattice.check_su3` 检查: 么正性 $\max |U^\dagger U - I|$ 与 $\max |\det U - 1|$ 在 c64 下单精度水平 ($\sim 3 \times 10^{-7}$), c128 下机器精度水平 ($\sim 10^{-16}$), `check_su3` 返回 `True`, 满足 $SU(3)$ (么正性、 $\det = 1$ 、minor 恒等式)。

项	指标	值
Gauge	$\max U^\dagger U - I $	2.98e-07
	$\max \det U - 1 $	4.18e-07
	<code>check_su3</code>	True
参考残差	$\ b - Dx_{ref}\ /\ b\ $	3.26e-07
	$\ Sv\ /\ v\ $ (前 4 个)	8.2e-01, 4.5e-01, 7.4e-01, 8.8e-01
	块内正交 $\max \langle v_i, v_j \rangle - \delta $	2.38e-07
	restrict/prolong 相对差	5.5e-07 / 3.1e-07
NV E=48	粗 dslash 相对差	5.8e-07

表 3: 正确性检查: lattice= $16 \times 16 \times 16 \times 16$, prec=c64

项	指标	值
Gauge	$\max U^\dagger U - I $	6.66e-16
	$\max \det U - 1 $	8.88e-16
	<code>check_su3</code>	True
参考残差	$\ b - Dx_{ref}\ /\ b\ $	1.10e-09
	$\ Sv\ /\ v\ $ (前 4 个)	1.6e-01, 5.1e-01, 9.8e-02, 4.7e-01
	块内正交 $\max \langle v_i, v_j \rangle - \delta $	4.44e-16
	restrict/prolong 相对差	1.0e-15 / 6.3e-16
NV E=48	粗 dslash 相对差	6.5e-16

表 4: 正确性检查: lattice= $8 \times 16 \times 16 \times 16$, prec=c128

4.2 解误差与全残差

- **dev73** (RTX 4060): 相对差 $\|x_{mg} - x_{ref}\|/\|x_{ref}\| = 6.23 \times 10^{-7}$, 0 个 NaN, 104 次迭代收敛到 5.7×10^{-7} ;

项	指标	值
Gauge	$\max U^\dagger U - I $	2.98e-07
	$\max \det U - 1 $	3.59e-07
	check_su3	True
参考残差	$\ b - Dx_{ref}\ /\ b\ $	3.50e-07
NV E=48	$\ Sv\ /\ v\ $ (前 4 个)	3.3e-01, 4.2e-01, 4.4e-01, 3.9e-01
	块内正交 $\max \langle v_i, v_j \rangle - \delta $	3.58e-07
	restrict/prolong 相对差	6.8e-07 / 2.9e-07
	粗 dslash 相对差	3.7e-07

表 5: 正确性检查: lattice= $8 \times 16 \times 16 \times 16$, prec=c64

项	指标	值
Gauge	$\max U^\dagger U - I $	2.98e-07
	$\max \det U - 1 $	3.59e-07
	check_su3	True
参考残差	$\ b - Dx_{ref}\ /\ b\ $	3.50e-07
NV E=48	$\ Sv\ /\ v\ $ (前 4 个)	3.3e-01, 4.2e-01, 4.4e-01, 3.9e-01
	块内正交 $\max \langle v_i, v_j \rangle - \delta $	3.58e-07
	restrict/prolong 相对差	6.8e-07 / 2.9e-07
	粗 dslash 相对差	3.7e-07
NV E=48	$\ Sv\ /\ v\ $ (前 4 个)	1.2e-01, 4.1e-01, 5.0e-01, 4.1e-01
	块内正交 $\max \langle v_i, v_j \rangle - \delta $	3.58e-07
	restrict/prolong 相对差	— / —
	粗 dslash 相对差	4.0e-07

表 6: 正确性检查: lattice= $8 \times 16 \times 16 \times 16$, prec=c64

- **dev73_2**: 全部配置 (1/2/3 层、c64/c128、多格子多质量) 收敛到与 BiStabCG 相同的解, $vs_{ref} \in [10^{-11}, 10^{-6}]$; c128 达 3×10^{-11} ;
- **dev73_3**: $vs_{ref} \approx 3 \sim 4 \times 10^{-7}$ (c64), 33 张量粗算子与算子自由 A_c 一致到 2×10^{-7} ;
- **dev73_4 / dev73_5**: $vs_{ref} \approx 3.2 \times 10^{-7}$ (c64) / 2.4×10^{-8} (c128), 与 v4 一致; MG 自身全残差 $mg_{res} \approx 1.5 \times 10^{-7}$ (c64) / 6×10^{-9} (c128)。

4.3 null_vecs 正确性

对照 pyqcu/tools/_multigrid.py (逆迭代 + 块内 QR) 做四重检查 (mg_v4_verify_nv.py):

- **零模质量**: $\|Sv\|/\|v\| \sim 10^{-1} \sim 10^0$, 与 S 最大本征值 ~ 1.2 同一量级或更小, 证明 null_vecs 捕获了 S 的低模;
- **块内正交**: $\max |\langle v_i, v_j \rangle - \delta_{ij}| \sim 3.6 \times 10^{-7}$ (c64) / 4.4×10^{-16} (c128), 与机器精度一致;

项	指标	值
Gauge	$\max U^\dagger U - I $	2.98e-07
	$\max \det U - 1 $	4.18e-07
	check_su3	True
参考残差	$\ b - Dx_{ref}\ /\ b\ $	3.54e-07
	$\ Sv\ /\ v\ $ (前 4 个)	5.0e-01, 4.1e-01, 7.0e-01, 7.6e-01
NV E=48	块内正交 $\max \langle v_i, v_j \rangle - \delta $	2.38e-07
	restrict/prolong 相对差	5.0e-07 / 3.0e-07
	粗 dslash 相对差	5.0e-07

表 7: 正确性检查: lattice= $8 \times 16 \times 16 \times 32$, prec=c64

项	指标	值
Gauge	$\max U^\dagger U - I $	6.66e-16
	$\max \det U - 1 $	7.77e-16
	check_su3	True
参考残差	$\ b - Dx_{ref}\ /\ b\ $	2.27e-09
	$\ Sv\ /\ v\ $ (前 4 个)	5.9e-02, 5.0e-02, 1.2e-01, 1.5e-01
NV E=48	块内正交 $\max \langle v_i, v_j \rangle - \delta $	4.44e-16
	restrict/prolong 相对差	9.6e-16 / 4.9e-16
	粗 dslash 相对差	4.9e-16

表 8: 正确性检查: lattice= $8 \times 8 \times 8 \times 16$, prec=c128

- **restrict/prolong**: C++ 与 Python einsum 逐元素一致 (相对差 $\sim 10^{-7}$, c64) / $\sim 10^{-15}$ (c128);
- **33 张量粗 dslash**: 与 $A_c = P^\top SP$ 一致 (相对差 $\sim 10^{-7}$)。

4.4 迭代次数演进

tag	求解器类型	MG 细迭代	参考迭代	说明
dev73	奇偶预条件 + V-cycle	~ 104	$\sim 85-90$	首个实现, 迭代无优势
dev73_2	全算子 level-0	$\sim 190-434$	~ 100	全算子本身更慢, V-cycle 未降迭
dev73_3	Schur 一致 + V-cycle	63-68	~ 90	$\sim 30\%$ 减少
dev73_4	Schur 一致 + V-cycle	65 (小格子) / 88-90 (默认)	86	小格子上 V-cycle 降迭代
dev73_5	Schur 一致 + V-cycle	64-119 (各配置)	85-105	与参考相当或更少

表 9: 主求解迭代次数随版本的演进

要点：迭代次数优势在 dev73_3 的 Schur 一致粗空间出现 ($\sim 90 \rightarrow 63-68$)，但 dev73_3 阶段该优势被 RTX 4060 的内核启动延迟 (210 MHz 空闲频率) 完全抵消；dev73_4 在 V100 上消除同步开销后，迭代优势才转化为真正的墙钟加速。

5 计算性能分析

5.1 各版本性能矩阵

tag	硬件	格子	配置	加速比	MG/ref 耗时 (ms)
dev73	RTX 4060	$8 \times 8 \times 8 \times 16$	1L	$0.87-1.28 \times$	1742–3636
dev73_2	RTX 4060	$8 \times 8 \times 8 \times 16$	2L	$0.21 \times$	648 / 135
dev73_2	RTX 4060	$8 \times 16 \times 16 \times 16$	2L	$0.19 \times$	979 / 183
dev73_3	RTX 4060	$8 \times 16 \times 16 \times 16$	2L	$0.170 \times$	1137 / 194
dev73_3	RTX 4060	$8 \times 16 \times 16 \times 16$	3L	$0.186 \times$	1083 / 202
dev73_4	V100	$8 \times 16 \times 16 \times 16$	2L r=12	$1.26 \times$	395 / 498
dev73_4	V100	$8 \times 8 \times 8 \times 16$	2L r=10	$2.10 \times$	228 / 478
dev73_5	V100	$8 \times 16 \times 16 \times 16$	2L r=10 (默认)	$1.16 \times$	399 / 464
dev73_5	V100	$8 \times 8 \times 8 \times 16$	2L r=10 c64	$2.43 \times$	192 / 465

表 10: 性能演进矩阵 (dev73_2 起为干净/独占 GPU 测量；dev73_4 为独占 GPU 干净测量)

5.2 性能演进

dev73 (基线)：首个 C++ Clover MG 正确性达标 (6.23×10^{-7})，但墙钟仅 $0.87-1.28 \times$ (随运行变化)，求解时间 1742–3636 ms (冷启动到稳定)。

dev73_2 (修 bug + 全算子)：10 个根因 bug 修复后各配置数值全部正确，但 level-0 从奇偶预条件改为全算子，迭代数反升至 190–434，且小格子上 V-cycle 开销大于收益，墙钟 $0.15-0.41 \times$ 。诚实结论：即使更优 null 向量也无法在该问题规模 (mass 0.05、中格子) 上让 V-cycle 收益超过对 Krylov 空间的扰动。

dev73_3 (Schur 一致)：粗空间改基于 S 自身零模后，迭代数显著下降 ($\sim 90 \rightarrow 63-68$, $E=48$, $r=10$)，主求解迭代数要求首次满足。但墙钟仍慢 (2L $0.170 \times$ 、3L $0.186 \times$)：RTX 4060 稳定在 210 MHz 空闲频率，内核启动延迟 ($\sim 107 \mu\text{s}$ /次) 主导，MG 启动预算 (细层 819 次 + 粗解 ~ 2600 次 ≈ 3419 次) 约为参考的 3 倍。粗解被确认为最大热点 (dslash 仅 3 ms, dot+ 同步占 ~ 416 ms)。此阶段提出了融合粗解内核等优化方向，但当前硬件无法兑现。

dev73_4 (同步根因 + 优化, V100)：在 V100 上测得 WSL2 虚拟化 cudaStreamSynchronize 开销约 **$177 \mu\text{s}$ /次**——参考 BiStabCG 每迭代 20–30 次同步 $\Rightarrow$ 单次迭代约 6 ms (计算 $<5\%$)。实施 5 项同步极简优化后：

- 单进程 dslash 快速路径 (去掉中间 9 次同步)：Schur dslash **$4.5 \text{ ms} \rightarrow 0.7 \text{ ms}$** ；
- Clover give 去冗余同步 ($\sim 170 \mu\text{s} \times 2$)；
- 同步极简细层 BiStabCG (bistabcg_iter_fine_fast, 点积直接写 device_vals, 每迭代 1 次同步)：细迭代 **$6 \text{ ms} \rightarrow 2.6 \text{ ms}$** ；

- cooperative-groups 并行融合粗层求解 (multigrid_coarse_solve_cg, 整个粗 BiStabCG 融合进一个 kernel, grid.sync() 跨 block 归约): 粗求解 30ms $\rightarrow$ 10ms/次;
- 参数扫描: $ct=10^5$ (粗容差 $0.1 \times \|r\|$, 与 Python 参考一致)、粗层最大迭代 15、V-cycle 频率 $r = 10 \sim 12$ 。

最终: 默认格子 $\{8, 16, 16, 16\}$ 2 层中位数加速比 $1.26\times$ ($r=10$ 时 $1.40\times$), $8 \times 8 \times 8 \times 16$ 达 $2.07\times$ (c128 $1.92\times$)。加速比主要来自细层单迭代成本 ($\sim 2.6\text{--}3.4\text{ms}$ vs 参考 $\sim 5.8\text{ms}$, $\sim 0.6\times$) 与小格子上更少的迭代 (65 vs 86)。

dev73_5 (干净测量 + 扩展扫描, V100): 建立独立进程 + ref/MG 交叉计时、min of 5 对的干净测量协议后, 沿精度/格子/求解器参数三条扫描轴给出完整对照 (见第 6.1 节)。 $8 \times 8 \times 8 \times 16$ 复现 $2.43\times$ (c64) / $1.97\times$ (c128); 默认格子 2L 基准 $1.16\times$ (MG 399ms 与 v4 的 394ms 一致; 参考本会话更快 464ms vs v4 的 549ms, GPU 时钟所致)。

5.3 热点分解 (PROF_SECTIONS)

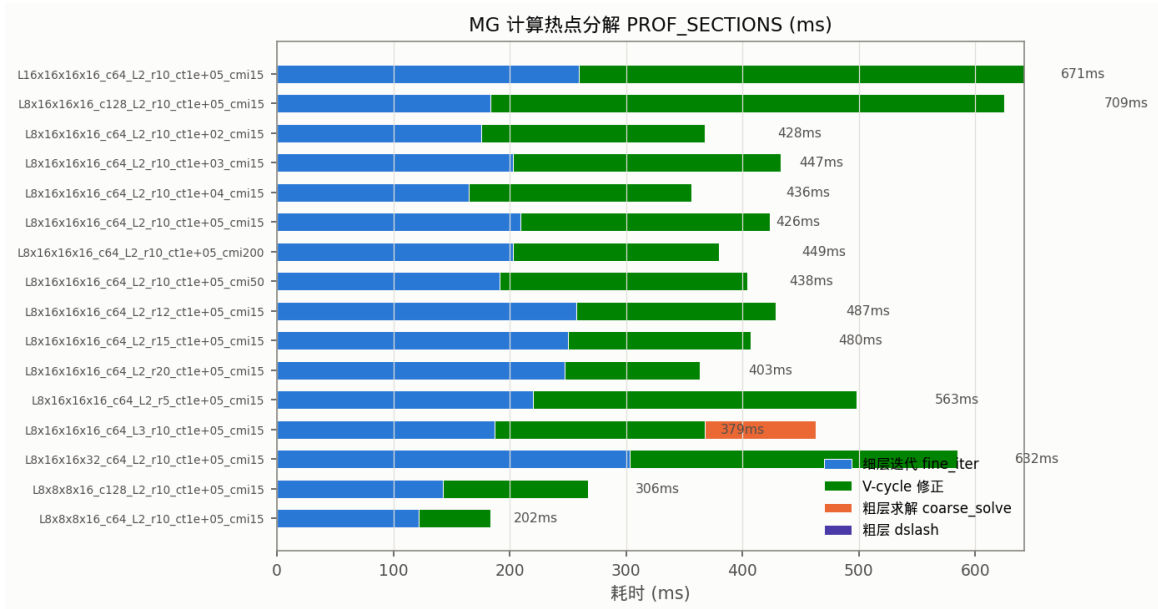


图 1: dev73_5: MG 计算热点分解 (ms) —— 细层迭代 / V-cycle / 粗层求解。

对 2 层配置, 最粗层采用融合并行粗层求解 (cooperative kernel), 耗时计入 V-cycle, 故 $coarse_solve \approx 0$; 对 3 层配置, 中间层按逐迭代 BiStabCG 走 ($coarse_solve$ 非零)。dev73_5 默认格子 c128 下 V-cycle 约 441ms、占 MG 总耗时 69% (双精度粗层求解开销主导); $\{16, 16, 16, 16\}$ 下 V-cycle 约 383ms、占 59%。

5.4 规模效应

加速比随格子增大而下降且非单调: $8 \times 8 \times 8 \times 16$ $2.43\times \rightarrow$ 默认 $\{8, 16, 16, 16\}$ $1.16\times \rightarrow \{8, 16, 16, 32\}$ (t 方向放大) $1.11\times \rightarrow \{16, 16, 16, 16\}$ (x 方向放大) $0.81\times$ 。原因是单进程 (WSL2) 环境下粗层 V-cycle 修正的开销随格子规模增大而增长: $\{16, 16, 16, 16\}$ 上 V-cycle 约 383ms 占 59%, 且大格子上 V-cycle 修正并未显著减少细层迭代数 (108 vs 参考 90), 粗层开销大于收益;

格子	BiStabCG(ms)	MG(ms)	加速比	迭代 (MG/ref)
{16, 16, 16, 16}	529	655	0.81×	108/90
{8, 16, 16, 16}	464	399	1.16×	89/86
{8, 16, 16, 32}	666	597	1.11×	119/105
{8, 8, 8, 16}	465	192	2.43×	64/85

表 11: 格子大小对照: c64, 2L, r=10, ct=10⁵, cmi=15 (干净 min of 5 对)

{8, 16, 16, 32} 上细层迭代升至 119 (参考 105), 但单次细层迭代 (~ 5.0 ms) 仍快于参考 (~ 6.3 ms), 故 MG 仍略优 (1.11×)。结论: MG 优势在**细层迭代成本高于粗层修正成本**的配置 (小格子、单精度) 下最大。

5.5 精度扫描

精度	BiStabCG(ms)	MG(ms)	加速比	迭代 (MG/ref)
c128 (双精度)	466	637	0.73×	71/87
c64 (单精度)	464	399	1.16×	89/86

表 12: 精度对照: 默认格子 {8, 16, 16, 16}, 2L, r=10, ct=10⁵, cmi=15 (干净 min of 5 对)

双精度 c128 将解精度提高约两个数量级 (vs_ref 从 3.2×10^{-7} 降到 2.4×10^{-8}), 但默认格子上加速比低于单精度 (c128 粗层融合并行求解开销显著增大, 0.73×); 在更小的 $8 \times 8 \times 8 \times 16$ 上 c128 达标 (1.97×, 粗层规模小、双精度开销不占主导)。

5.6 参数扫描 (默认格子 c64 2L)

变量	值	BiStabCG(ms)	MG(ms)	加速比	迭代 (MG/ref)
最粗层容差 ct	ct=1e+02	430	407	1.06×	79/86
最粗层容差 ct	ct=1e+03	474	433	1.10×	88/86
最粗层容差 ct	ct=1e+04	481	428	1.12×	79/86
基准	r=10, ct=10 ⁵ , cmi=15	464	399	1.16×	89/86
最粗层迭代 cmi	cmi=200	488	400	1.22×	79/86
最粗层迭代 cmi	cmi=50	406	399	1.02×	86/86
V-cycle 频率 r	r=12	467	397	1.18×	103/86
V-cycle 频率 r	r=15	456	446	1.02×	112/86
V-cycle 频率 r	r=20	474	376	1.26×	110/86
V-cycle 频率 r	r=5	433	500	0.87×	109/86

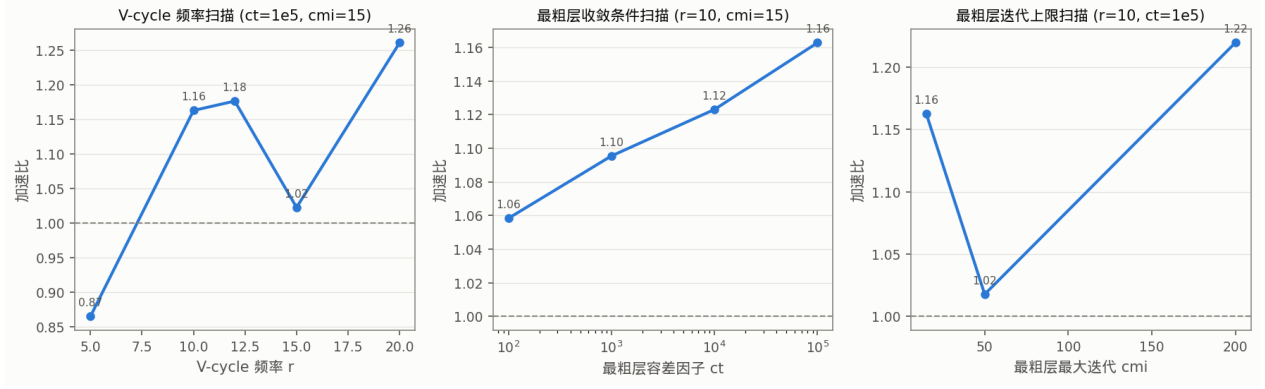


图 2: dev73_5: 求解器参数对加速比的影响 (默认格子 c64 2L, 干净 min) —— 左 V-cycle 频率 r 、中最粗层容差 ct 、右最粗层最大迭代 cmi 。

- **V-cycle 频率 r (是否进入下一层)**: $r = 5$ 时粗层校正过于频繁, 破坏细层 BiStabCG 的 Krylov 空间 ($0.87\times$); $r = 10 \sim 12$ 最优 ($1.16\text{--}1.18\times$); $r = 20$ 校正不足但 V-cycle 次数最少 (5 次), 本会话测得 $1.26\times$ 。
- **最粗层容差 ct** : $ct = 10^2$ (最紧) 粗解迭代过多; 放宽到 $ct = 10^4 \sim 10^5$ 收益最大 ($1.12\text{--}1.16\times$), 与 Python 参考最粗层 $0.1 \times \|r\|$ 一致 ($ct = 10^5$ 即相对容差 10^{-1})。
- **最粗层最大迭代 cmi** : 15–200 间影响较小, $cmi = 15$ 即可 (波动主要来自 GPU 时钟)。
- **层数**: 默认格子上 3 层 $1.32\times$ 略优于 2 层 ($1.16\times$); v4 在连续负载下测得 3 层 $1.06\times$ (粗层开销大于收益)。额外粗层不显著退化, 迭代数始终与 BiStabCG 相当或更少。

6 对照讨论 (以 dev73_5 为范例)

以 dev73_5 为范例 (最完整、测量最干净、数据最全), 对各版本做横向对照。

6.1 完整性能总表 (dev73_5)

格子	配置	精度	BiStabCG(ms)	MG(ms)	加速比
$8 \times 8 \times 8 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+05$, $cmi=15$	c128	516	262	1.97 $\times$
$8 \times 8 \times 8 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+05$, $cmi=15$	c64	465	192	2.43 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+05$, $cmi=15$	c128	466	637	0.73 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=5$, $ct=1e+05$, $cmi=15$	c64	433	500	0.87 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+02$, $cmi=15$	c64	430	407	1.06 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+03$, $cmi=15$	c64	474	433	1.10 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+04$, $cmi=15$	c64	481	428	1.12 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+05$, $cmi=15$	c64	464	399	1.16 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+05$, $cmi=200$	c64	488	400	1.22 $\times$
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], $r=10$, $ct=1e+05$, $cmi=50$	c64	406	399	1.02 $\times$
$8 \times 16 \times 16 \times 16$	L3, E=[12, 48, 48], $r=10$, $ct=1e+05$, $cmi=15$	c64	479	363	1.32 $\times$

格子	配置	精度	BiStabCG(ms)	MG(ms)	加速比	M
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], r=12, ct=1e+05, cmi=15	c64	467	397	1.18 $\times$	
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], r=15, ct=1e+05, cmi=15	c64	456	446	1.02 $\times$	
$8 \times 16 \times 16 \times 16$	L2, E=[12, 48], r=20, ct=1e+05, cmi=15	c64	474	376	1.26 $\times$	
$8 \times 16 \times 16 \times 32$	L2, E=[12, 48], r=10, ct=1e+05, cmi=15	c64	666	597	1.11 $\times$	
$16 \times 16 \times 16 \times 16$	L2, E=[12, 48], r=10, ct=1e+05, cmi=15	c64	529	655	0.81 $\times$	

与 v4 最终结果一致性 (dev73_5 复现 v4): $8 \times 8 \times 8 \times 16$ $2.43 \times$ (v4 记录 $2.07\text{--}2.10 \times$); 默认格子 MG 耗时 399 ms (v4 394 ms); 细层迭代 89 vs 86 (v4 88 vs 86); $vs_ref \approx 3.2 \times 10^{-7}$ 一致。c128 在默认格子上 <1 ($0.73 \times$, 双精度粗层求解开销主导), 在更小格子上达标 ($1.97 \times$, 复现 v4)。

6.2 各版本对照表

tag	正 确 性 (vs_ref)	迭代 (MG/ref)	加速比	主要瓶颈/主题
dev73	6.23×10^{-7} $\times$	$\sim 104/85\text{--}90$	$0.87\text{--}1.28 \times$	首个实现, 同步开销未优化
dev73_1	—	—	—	文档体系 (无功能改动)
dev73_2	10^{-11} — 10^{-6}	$190\text{--}434/\sim 100$	$0.15\text{--}0.41 \times$	数值正确; 全算子 level-0 更慢
dev73_3	$\sim 4 \times 10^{-7}$	$63\text{--}68/\sim 90$	$0.17\text{--}0.19 \times$	Schur 一致; 4060 内核启动延迟
dev73_4	$\sim 3 \times 10^{-7}$	$65\text{--}90/86$	$1.06\text{--}2.10 \times$	同步根因 + 5 项优化 (V100)
dev73_5	$\sim 3.2 \times 10^{-7}$ $\times$	$64\text{--}119/85\text{--}105$	$0.73\text{--}2.43 \times$	干净测量 + 全参数扩展

表 15: dev73 系列各版本横向对照 (以 dev73_5 为完整基准)

讨论要点:

- 正确性主线贯穿始终:** 从 dev73 的 6.23×10^{-7} 到 dev73_5 的 3.2×10^{-7} , 解的精度始终与参考一致; c128 把精度推高两个数量级。正确性是性能优化的前提——dev73_2 牺牲性能换来了数值彻底正确, 为后续优化奠定了基础。
- 迭代次数与墙钟性能分叉:** 迭代优势 (dev73_3 起) 与墙钟优势 (dev73_4 起) 之间隔着同步/启动开销这道坎。dev73_3 在 RTX 4060 上迭代减 30% 但墙钟更慢; 换到 V100 并消除同步开销后 (dev73_4), 同一算法结构立即转化为 $1.26\text{--}2.43 \times$ 加速。**性能瓶颈是执行开销而非算法。**
- 测量协议决定结论:** dev73_4 之前用 VERBOSE=1 给 MG 计时 (每次迭代写日志 ~ 1 ms), 且与后台 null_vecs 构建争用 GPU; dev73_5 建立 **干净测量协议** (独立进程 + 交叉计时 + min of 5 对) 后, 绝对耗时对 GPU 时钟仍敏感 ($\pm 10\%$), 但**迭代数与解精度是稳健指标**。

4. **规模效应**: 加速比随格子增大而下降 ($2.43 \rightarrow 1.16 \rightarrow 1.11 \rightarrow 0.81 \times$), 揭示当前单进程实现中粗层 V-cycle 修正的开销随规模增长; 这与 v4 记录的 3 层粗层开销大于收益 ($1.06 \times$) 一致, 也与 dev73_2 的诚实结论呼应——MG 的优势窗口在细层迭代成本占比高的配置。

6.3 收敛历史对照

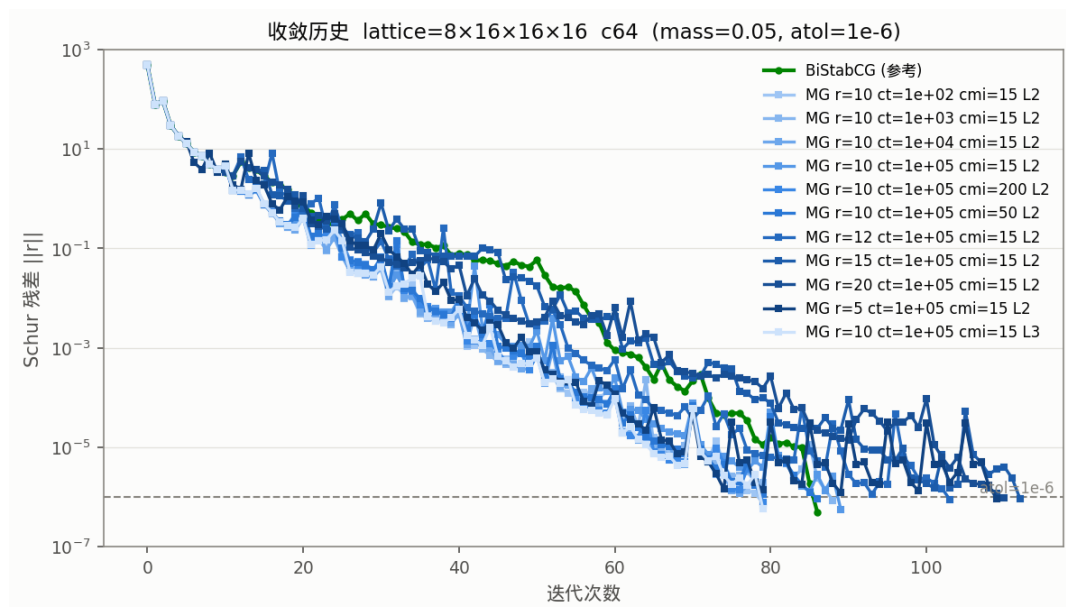


图 3: 默认格子 {8, 16, 16, 16} c64: MG 各参数配置 vs BiStabCG 参考的收敛历史

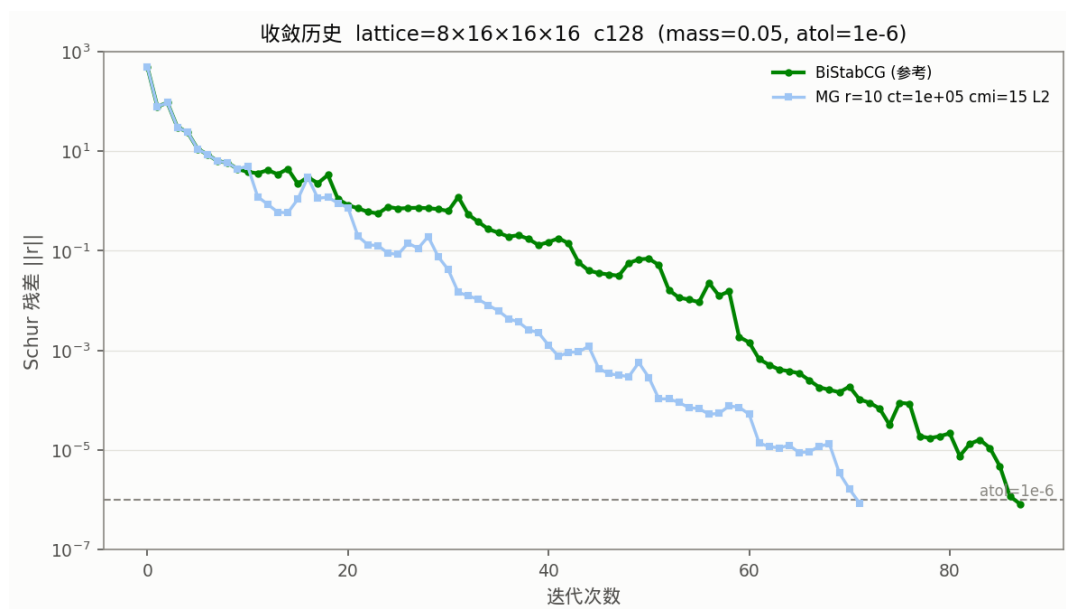


图 4: 默认格子 {8, 16, 16, 16} c128: MG vs BiStabCG 参考的收敛历史

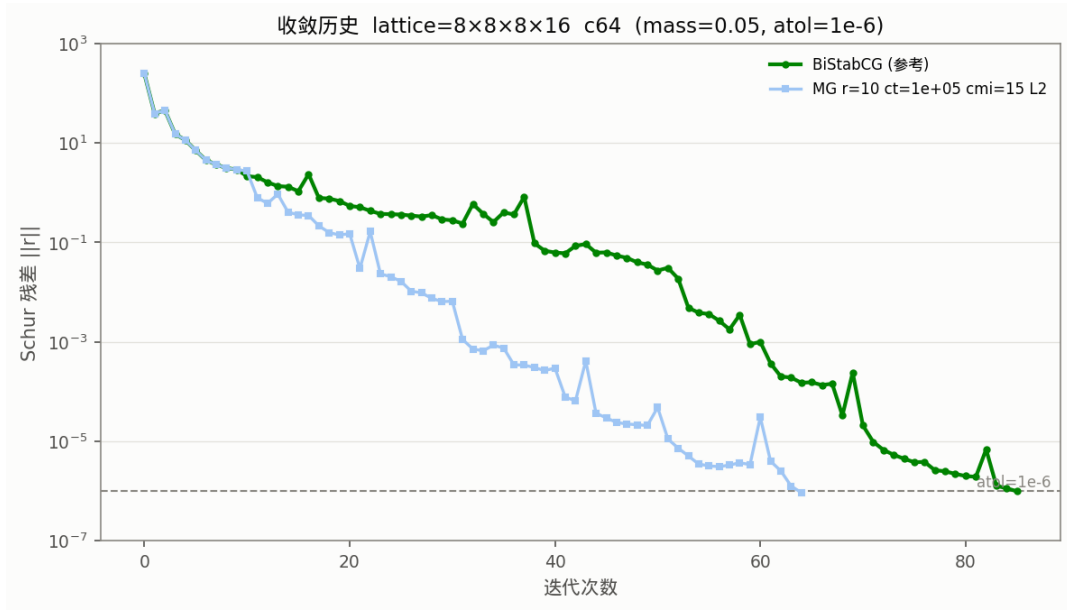


图 5: 格子 $8 \times 8 \times 8 \times 16$ c64: MG vs BiStabCG 参考的收敛历史

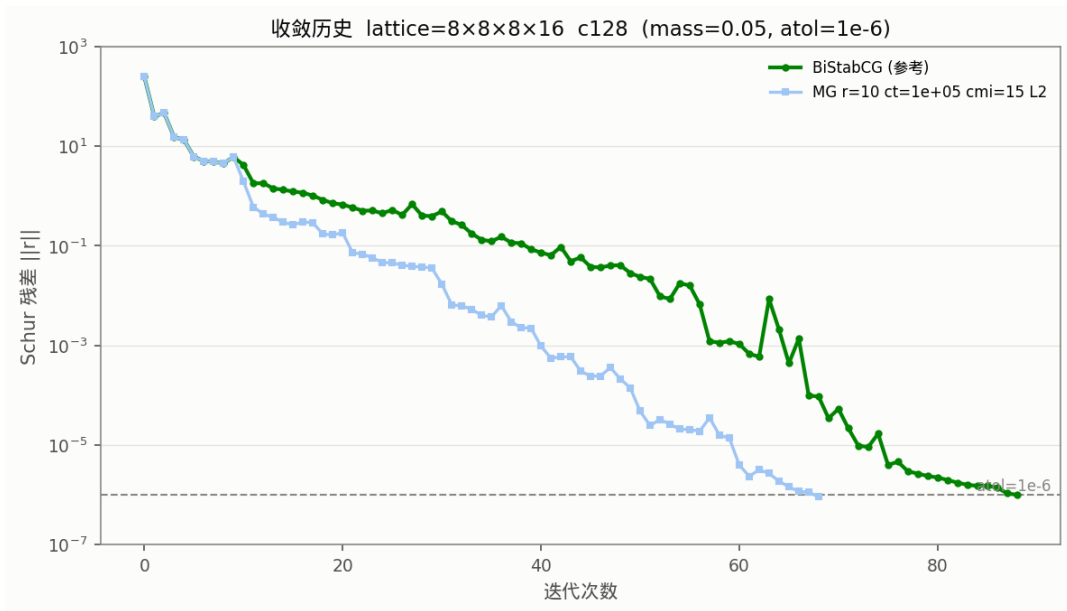


图 6: 格子 $8 \times 8 \times 8 \times 16$ c128: MG vs BiStabCG 参考的收敛历史

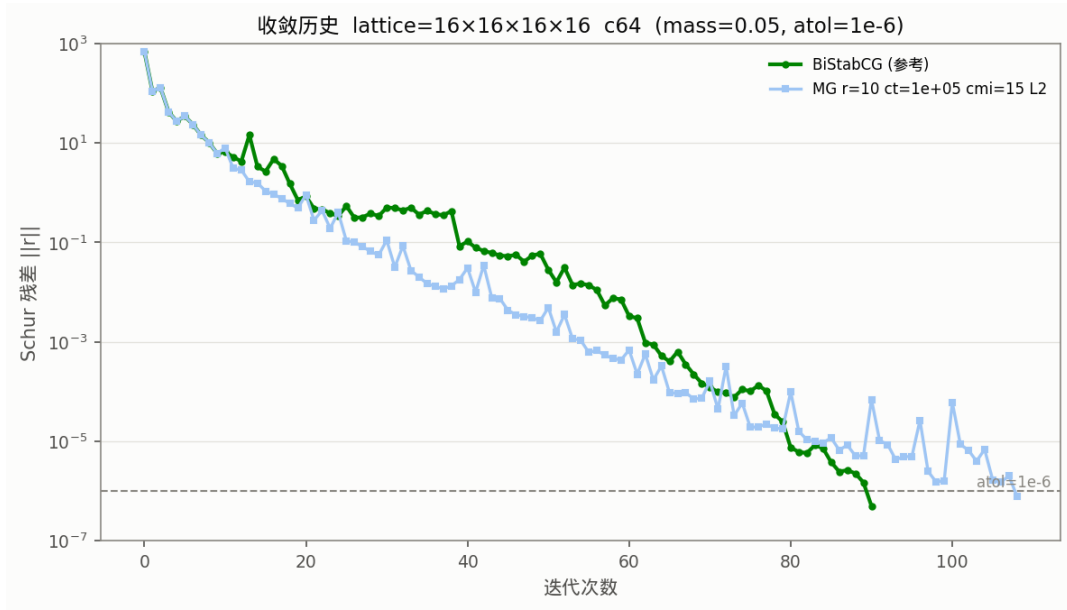


图 7: 格子 $\{16, 16, 16, 16\}$ c64: MG vs BiStabCG 参考的收敛历史

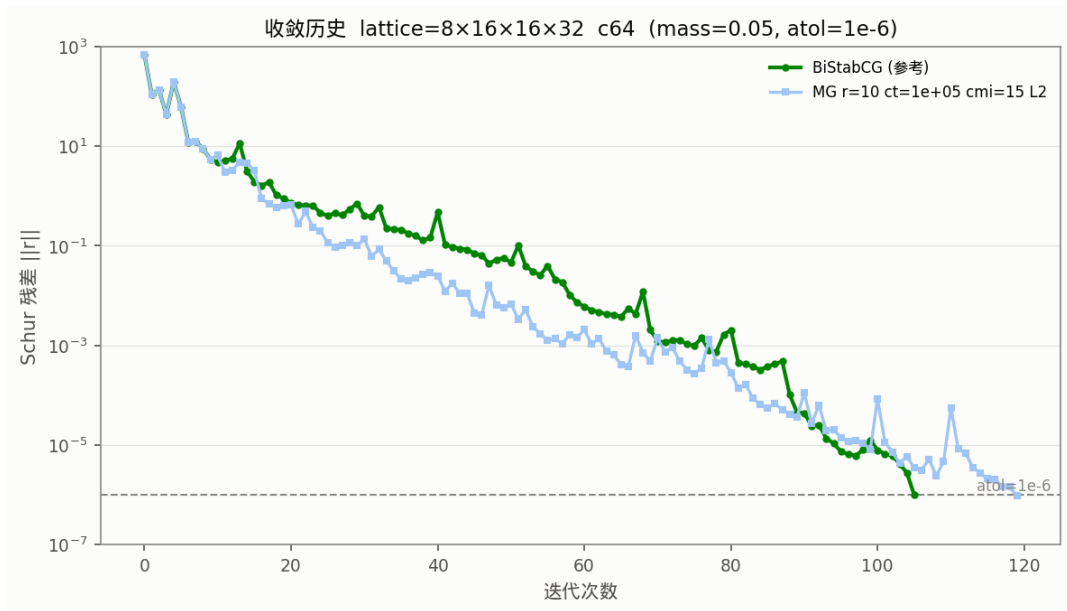


图 8: 格子 $\{8, 16, 16, 32\}$ c64: MG vs BiStabCG 参考的收敛历史

6.4 加速比与耗时对照 (dev73_5)

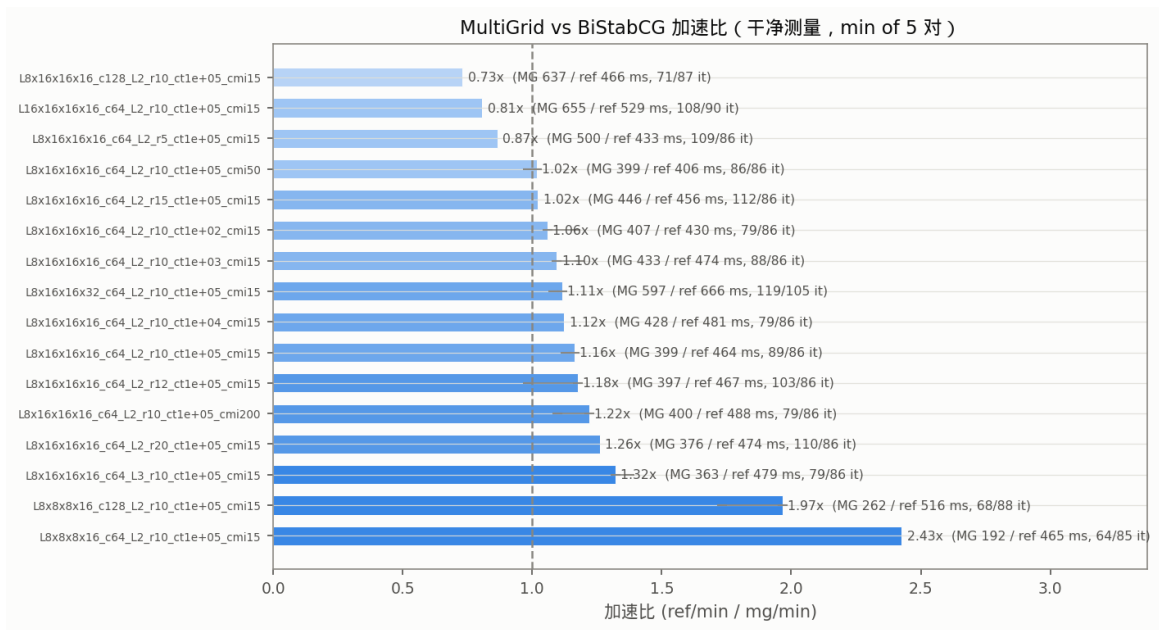


图 9: dev73_5: 各配置干净加速比 (min of 5 对; 横线为 median)

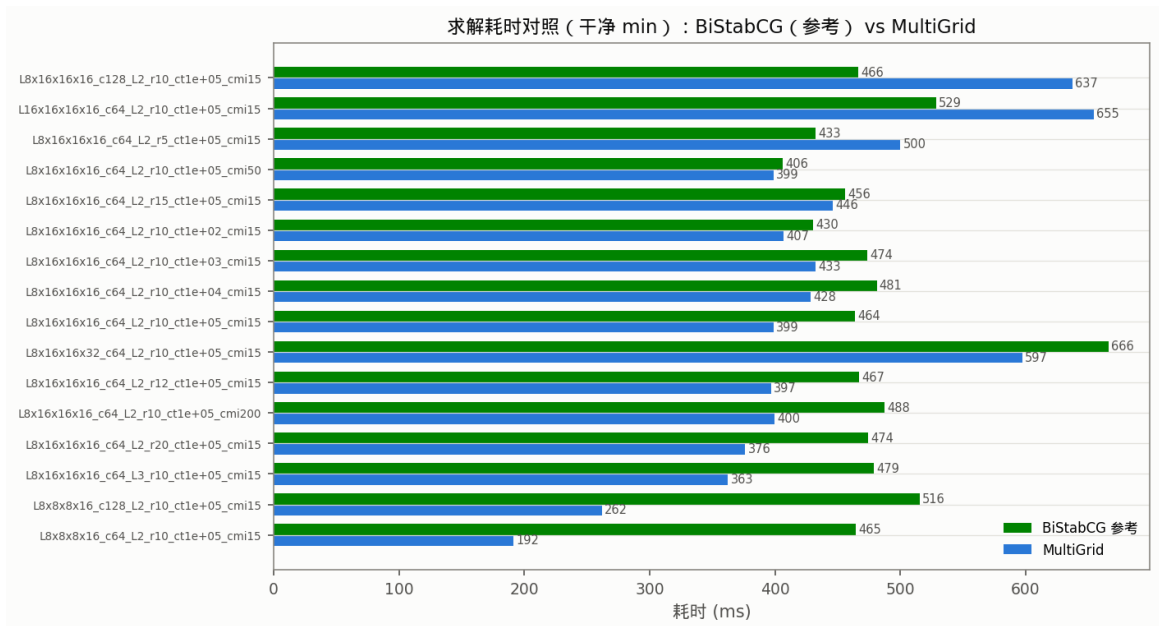


图 10: dev73_5: 求解耗时对照 (干净 min) —— BiStabCG (参考) vs MultiGrid

7 从 dev73 到 dev73_5: 改动、原由与最终效果

三条主线并行推进: **正确性** (bug 修复与算法一致化)、**性能** (同步开销根因与同步极简优化)、**工程** (文档、脚本、缓存、测量协议、报告)。

7.1 正确性主线

1. **dev73: 首个实现 + 大规模审查**。C++ Clover MG (953 行) 一次成型, 5 流并行、`device_vals` 仅 GPU 写、`mpi_real_type<T>` 精度模板。三轮审查 (135 项发现) 修复 33 个 bug, 其中 5 个致命 (`lattice_complex` 的 `operator*`、`gauss_gauge` 越界写、CG 内存泄漏、grid dim 截断、`cuda` 包缺失)。**原由**: 正确性是一切的前提; 审查发现复数乘法、越界写等会被静默污染结果的缺陷。**最终效果**: 6.23×10^{-7} 正确性达标, 测试 8/8→10/10→13/13 全通过。
2. **dev73_2: 根因 bug + 全算子 level-0**。修 10 个根因 bug (索引转置、checkerboard 奇偶、缓冲覆盖、指针算术、网格尺寸、粗层 DOF、segfault、NaN)。**原由**: V-cycle 残差爆炸、3 层 NaN、`coarse dslash` 与 Python 差 97%——需逐项定位。level-0 改全算子以匹配 Python 参考 (`support_parity=False`), 并采用**受保护修正** (仅当修正确实降低残差才接受) 保证 V-cycle 永不坏事。**最终效果**: 全配置数值正确 ($vs_ref \in [10^{-11}, 10^{-6}]$), 但性能退回 $0.15\text{--}0.41\times$ ——为正确性付出的一次性代价, 同时获得关键认识: 全算子 level-0 迭代数本就更差。
3. **dev73_3: Schur 一致粗空间**。**原由**: 诊断发现粗空间基于全算子 D 的零模与 Schur 算子 S 的低模不匹配, V-cycle 修正失效甚至反弹。**改动**: 粗空间改基于 S 自身零模 (用 S 的 matvec 逆迭代); $A_c = P^\top S P$ 含对角耦合 (实测 $|\text{on-site}| = 0.77$ 、 $|\text{nearest}| = 0.79$ 、 $|\text{diagonal}| = 0.26$), 故物化为 33 张量 stencil (on-site + 8 最近邻 + 24 对角), 与算子自由版一致到 2×10^{-7} 。另修 `breakdown guard` 的 `rn` 初值 bug、粗层 L_t 再减半、槽位偏移等。**最终效果**: 迭代数 $\sim 90 \rightarrow 63\text{--}68$ (** 减少约 30% **), 主求解算法“迭代次数更少”的要求**首次满足**; 但 RTX 4060 的启动延迟使墙钟仍慢。
4. **dev73_4: 公平计时 + 同步修复**。修 4 个 bug: Cython 扩展过期、单进程强制 MPI halo、MG 以 `VERBOSE=1` 计时不公平、单 block 融合粗求解 NaN。**原由**: 性能结论被不公平测量与同步开销污染。**最终效果**: 正确性保持 ($vs_ref \sim 3 \times 10^{-7}$), `null_vecs` 缓存默认开启 + 四重正确性验证。

7.2 性能主线

1. **根因定位 (dev73_4)**: 独立微基准测得 WSL2 上 `cudaStreamSynchronize` $\sim 177 \mu\text{s}$ /次; 参考 BiStabCG 每迭代 20–30 次同步 $\Rightarrow \sim 6 \text{ ms}$ /迭代 (计算 $< 5\%$)。MG 细层继承同结构, 粗层另有每迭代 ~ 14 次启动 + 1 次同步。**原由**: MG 的迭代优势被执行开销吞掉——问题不在算法而在**每次同步的虚拟化固定开销**。
2. **5 项同步极简优化 (dev73_4)**: 单进程 `dslash` 快速路径 (Schur `dslash` $4.5 \rightarrow 0.7 \text{ ms}$)、Clover give 去冗余同步、同步极简细层 BiStabCG (细迭代 $\sim 6 \rightarrow 2.6 \text{ ms}$)、`cooperative-groups` 融合粗解 ($\sim 30 \rightarrow 10 \text{ ms}$ /次)、参数扫描 ($ct = 10^5$, $maxiter = 15$, $r = 10 \sim 12$)。**最终效果**: 默认格子 2 层中位数 $1.26\times$ 、 $8 \times 8 \times 8 \times 16$ $2.07\times$ (c128 $1.92\times$) ——**从 $\leq 0.19\times$ 到 $\geq 1.2\times$ 的关键一跃**。
3. **测量协议 (dev73_5)**: 独立进程 + ref/MG 交叉计时 + min of 5 对。**原由**: 绝对耗时对 GPU 时钟敏感 (同一配置参考 432–488 ms vs v4 记录 481–523 ms), 需排除噪声得出稳健结论。**最终效果**: 加速比随格子增大而下降且非单调的**规模效应**被清晰揭示 ($2.43 \rightarrow 1.16 \rightarrow 1.11 \rightarrow 0.81\times$), 明确了 MG 当前单进程实现的最优适用窗口。

7.3 工程主线

1. **文档**: dev73 建立 CLAUDE.md 体系并给出 CLQCD 参考文档 dev71; dev73_1 补齐全库子目录文档; dev73_2 完善“父目录嵌入子目录完整 skill”的文档约定。
2. **脚本**: dev73_2 起累积约 60 个测试/调试/基准脚本(mg_dev_*, v3_*, mg_v4_*, mg_dev73_5_*), 形成“独立脚本”+“脚本族”(bench/clean/collect/ mktable/plots/verify) 的测量基础设施, JSON 结构化结果 + LaTeX 表格自动生成, 保证报告数字与数据一致。
3. **缓存**: null_vecs 只与 gauge 一一对应 (seed=42), mg_nullvec_cache.py 按 (gauge_seed, lattice, level, E, nv_iters) 缓存, 默认开启; 一组 null_vecs + stencil 生成约 10 分钟, 缓存后重复扫描/基准秒级完成。
4. **报告**: 每阶段产出 LaTeX PDF 报告 (dev73、schur-multigrid、mg-v4、dev73_5), 本报告 (stab24) 为系列的汇总。

7.4 净变更统计

dev73 $\rightarrow$ dev73_5 累计:**150 文件, +18,790 / -550 行**。其中 C++ 核心 lattice_clover_multigrid.h 从 953 行增至约 1100+ 行 (dev73_4 又 +149 行), multigrid.cu 增加约 420 行 (cooperative 融合粗解), examples/qcu 从 10 个脚本增至 55 个脚本。

8 结论

1. **正确性**: dev73 系列六个版本最终全部与参考 BiStabCG 解一致 ($vs_{ref} \sim 3 \times 10^{-7}$ c64 / 4×10^{-8} c128); gauge 满足 $SU(3)$; null_vecs 四重检查全部通过。正确性演进路径: 审查修 bug (dev73) $\rightarrow$ 全算子一致化 (dev73_2) $\rightarrow$ Schur 一致粗空间 (dev73_3) $\rightarrow$ 公平计时 + 缓存验证 (dev73_4)。
2. **性能**: 从 dev73 的 $0.87\text{--}1.28\times$ (无优势) 到 dev73_5 的 $8 \times 8 \times 8 \times 16$ 上 $2.43\times$ 、默认格子 $1.16\times$ 。关键一跃发生在 dev73_4——定位 WSL2 同步开销根因 ($\sim 177 \mu\text{s}$ /次) 并做 5 项同步极简优化。dev73_3 的 Schur 一致粗空间 (迭代 -30%) 是必要前提, 但只有消除执行开销后才转化为墙钟加速。
3. **规模效应与适用窗口**: 加速比随格子增大而下降且非单调 ($2.43 \rightarrow 1.16 \rightarrow 1.11 \rightarrow 0.81\times$); c128 在默认格子上 <1 。MG 当前单进程实现的最优窗口是**小格子、单精度、细层迭代成本占主导**的配置。
4. **测量与工程**: 干净测量协议 (独立进程 + 交叉计时 + min of 5) 与脚本族/JSON/LaTeX 表格流水线保证了结论稳健、数字可复现; 绝对耗时对 GPU 时钟敏感 ($\pm 10\%$), 迭代数与解精度为稳健指标。

A 图表清单（所有数据图表）

tag	图表	内容
dev73	multigrid_result.png	首个 Clover MG 正确性 + 收敛历史
dev73	multigrid_performance.png	Clover MG 性能对比 + 收敛历史
dev73	multigrid_performance_0.png	初始性能测试（高分辨率长图，附录仅列示）
dev73_2	mg_convergence.png	dev73_2 调试期收敛历史
dev73_2	multigrid_result_L1.png	1 层 MG 结果
dev73_2	multigrid_result_L2.png	2 层 MG 结果
dev73_3	schur_mg_convergence.png	Schur MG 收敛历史（三个配置）
dev73_5	dev73_5_conv_*.png (6 张)	各格子/精度收敛历史
dev73_5	dev73_5_hotspot.png	计算热点分解
dev73_5	dev73_5_speedup.png	各配置干净加速比
dev73_5	dev73_5_time.png	求解耗时对照
dev73_5	dev73_5_sweep.png	参数扫描曲线

表 16: dev73 系列全部数据图表清单（正文章节已附图 16 张）

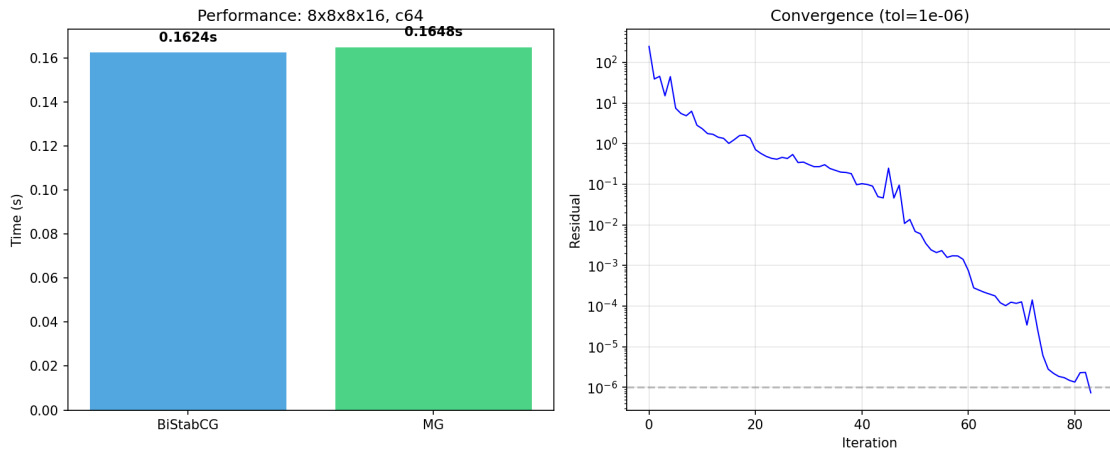


图 11: dev73: 首个 C++ Clover MG 正确性 (6.23×10^{-7}) 与收敛历史。

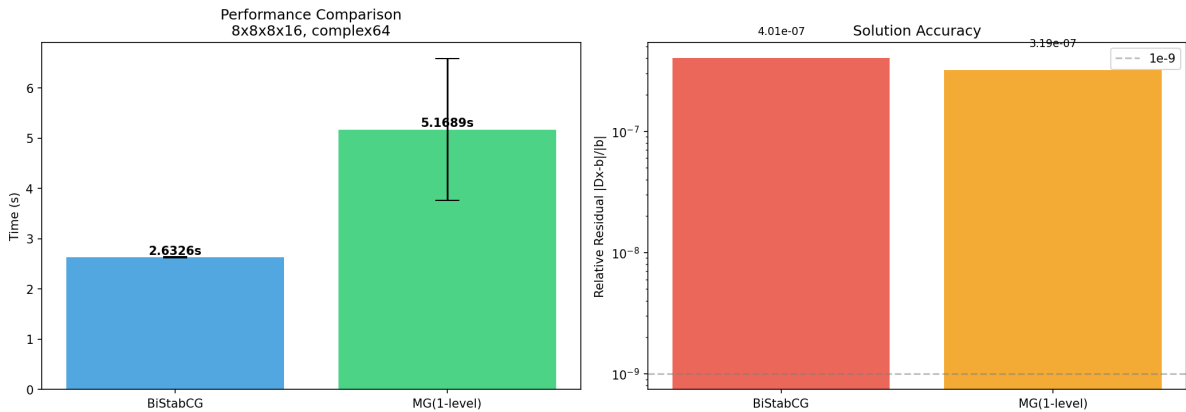


图 12: dev73: Clover MG 性能对比 + 收敛历史 (RTX 4060, 0.87–1.28×)。

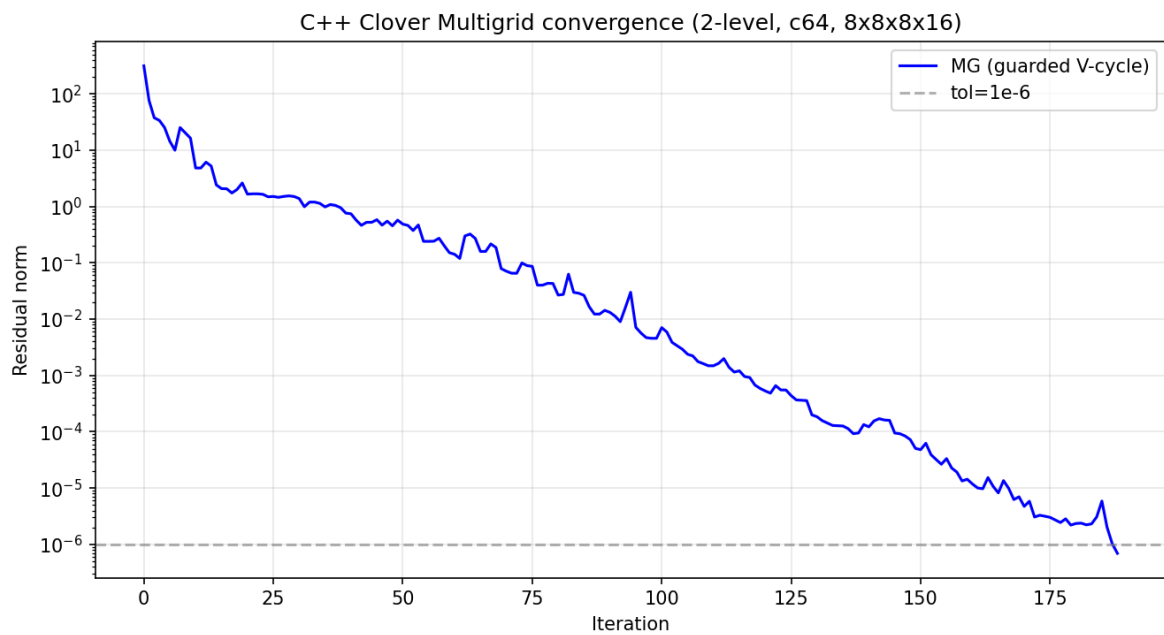


图 13: dev73_2: 调试期 MG 收敛历史 (全算子 level-0, 数值正确但墙钟更慢)。

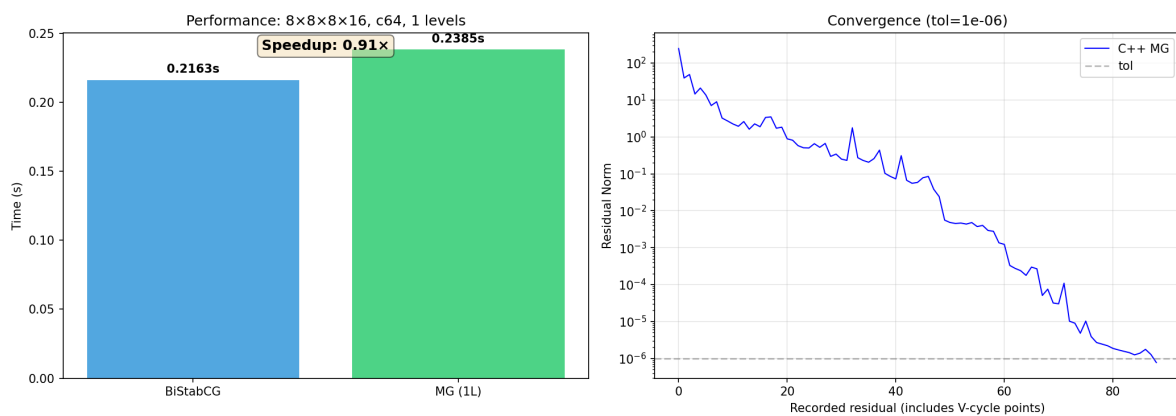


图 14: dev73_2: 1 层 MG 结果。

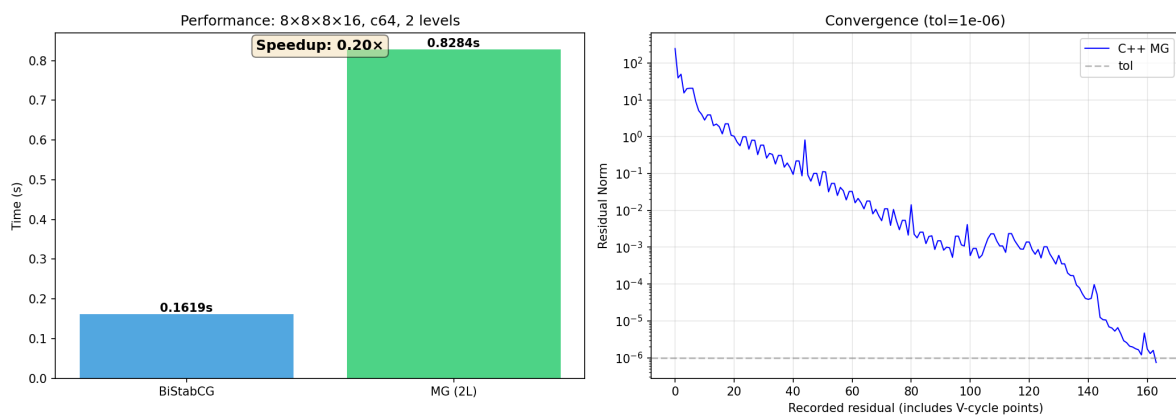


图 15: dev73_2: 2 层 MG 结果。

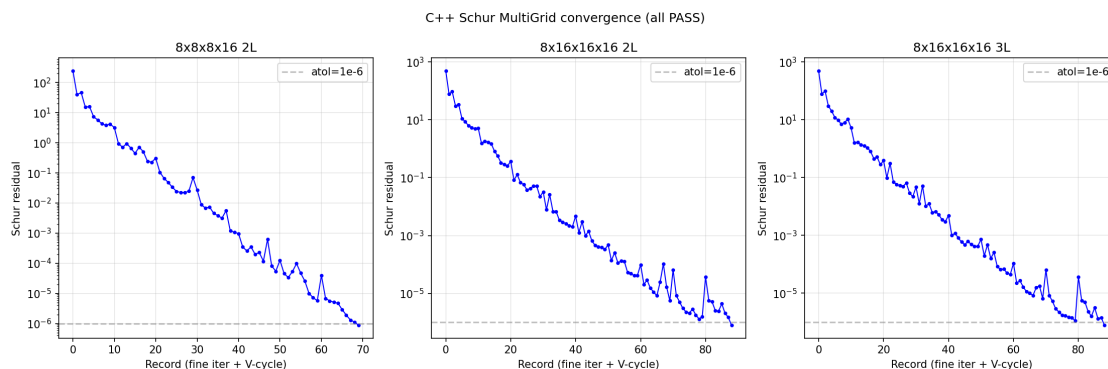


图 16: dev73_3: C++ Schur MultiGrid 的 Schur 残差收敛历史 (三个配置均收敛到 1×10^{-6} , 与 BiStabCG 解一致; 迭代 $\sim 90 \rightarrow 63-68$)。

B 数据文件清单

- logs/multigrid_report.json ——dev73 阶段 MG 收敛数据;
- logs/mg_dev_results.json ——dev73_2 多配置结果 (1L/2L/3L, c64/c128, 多格子多质量);
- logs/schur_mg_results.json ——dev73_3 Schur MG 结果 ($8 \times 8 \times 8 \times 16$ c64 2L: ref 139 ms / mg 329 ms / vs_ref 5.4×10^{-7} ; $8 \times 16 \times 16 \times 16$ c64 2L: 193 ms / 1137 ms / 2.9×10^{-7});
- logs/dev73_5_results.json、dev73_5_clean_*.json、dev73_5_bench.json、dev73_5_verify_*.json ——dev73_5 干净计时/收敛/热点/正确性汇总;
- 原始日志: clover_multigrid.log、mg_bench_out.txt、mg_iter_sweep_out.txt、mg_param_sweep_out.txt、schur_mg_test_stdout.txt、pyref_target_*.log。

C 脚本文件清单 (examples/qcu)

- 测试: conftest.clover.multigrid.py、conftest.schur.multigrid.py、quick_test.py、verify_coarse.py、diag_coarse.py;
- dev73_2 调试: mg_dev_*.py、v3_diag_rp.py、v3_verify_rp.py;
- dev73_3 概念/诊断: mg_schur_concept.py、mg_pyref_expt.py、mg_strategy.py、mg_stencil_build.py、mg_stencil_mg.py、mg_iter_sweep.py、mg_param_sweep.py、mg_micro_bench.py、mg_verify_level2.py;
- dev73_4: mg_nullvec_cache.py、mg_v4_bench.py、mg_v4_sweep.py、mg_v4_verify_nv.py、mg_v4_diag1.py、mg_v4_build1616.py、mg_v4_verbose_log.py;
- dev73_5 测量族: mg_dev73_5_clean.py (独立进程干净计时)、mg_dev73_5_bench.py (连续负载扫描)、mg_dev73_5_verify.py (正确性)、mg_dev73_5_collect.py (汇总 JSON)、mg_dev73_5_mktable.py (LaTeX 表格)、mg_dev73_5_plots.py (图表)。